

Advanced Web Programming

SIB245007 | D-IV Sistem Informasi Bisnis

Meeting 7: **Authentication, Authorization, and RBAC**

Who you are, what you're allowed to do, and how to check both on every request

What You'll Learn

1. Tell apart **authentication** (proving identity) from **authorization** (deciding permission), and why the two are kept separate
2. Understand how **sessions**, **cookies**, and related features (remember me, password reset, login rate limiting) make login feel safe and convenient
3. Recognize the basic rules for storing **passwords** and **RBAC** (role-based access control) through **middleware**, **Gates**, and **Policies**

This deck covers concepts. Implementing login and role restrictions for Simple POS happens in the practicum jobsheet.

Part 1

Authentication: Proving Identity

Two Different Questions

Authentication

- The question: "who are you?"
- Proven with an email and password
- The result: the user's identity is known

Authorization

- The question: "what are you allowed to do?"
- Checked once identity is known
- The result: a specific page or action is allowed or denied

An employee whose badge is valid at the lobby can still be turned away at the server room. The lobby checks identity, the server room checks permission.

Why HTTP Needs Sessions

Stateless: every HTTP request stands on its own. The server does not automatically remember earlier requests, even from the same browser.

- Without an extra mechanism, every page looks like it was opened by a stranger
- A session gives the server a way to remember the user between requests, without asking them to log in again on every page

The Login Flow with a Session



- Later requests carry the same cookie, the server looks up its session, and knows which user is asking
- The cookie holds only the session ID, the user's data stays on the server

Cookies and the Session ID

Session: the data of a logged-in user, stored on the server and identified by one random ID that is sent in a cookie.

- The session ID must be hard to guess, because anyone who holds it can pose as that user
- A sensitive cookie should carry the `HttpOnly` flag (JavaScript cannot read it) and the `Secure` flag (sent only over HTTPS)

Remember Me: Login That Outlasts the Session

Remember Me: an extra, long-lived cookie (weeks, not minutes) that keeps a user logged in even after their session on the server has ended, for example after closing the browser.

- This cookie carries a separate random token, not the regular session and not the password, stored in a column like `remember_token` on the users table
- When the user comes back, the server matches that token, creates a new session, then replaces it with a fresh token again
- Because it lasts so long, checking "Remember Me" should be the user's own choice, not the default behavior

Session vs Token

Session (the server stores state)

- The server keeps the login status
- Revoking access means deleting the session
- Suits web applications with pages in a browser

Token (the client stores it)

- The client carries a signed proof of identity
- The server does not need to store the login status
- Suits APIs used by many clients, covered in more depth later

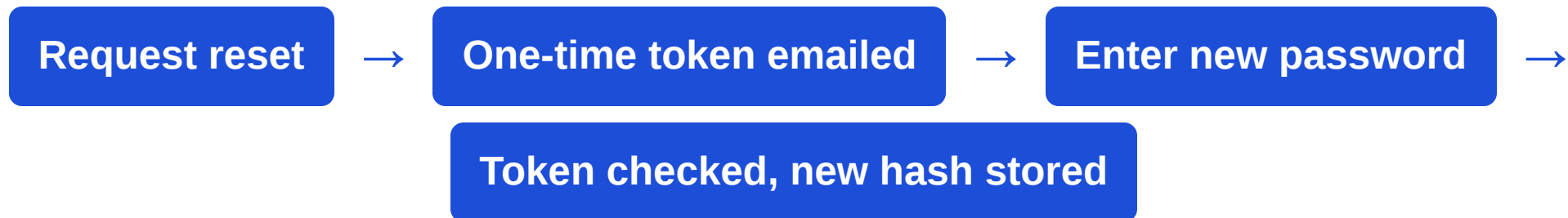
Passwords Are Never Stored in Plain Text

Hash: the output of a one-way function applied to a password plus a random salt, so the original password cannot be read back from the database.

- If the database leaks, the attacker gets hashes, not passwords they can use directly
- At login, the server hashes the typed password and compares it with the stored hash

Forgot Password

Because passwords are stored as hashes, the server itself cannot read one back to resend by email. The fix isn't sending the old password, it's giving the user a chance to set a new one:



- The reset token must expire quickly and work only once, just like a session ID that gets regenerated after login
- The reset link is emailed, not shown directly on the page, so only the real owner of that email can use it

Why Password Hashing Is Deliberately Slow

- Fast hashing lets an attacker try billions of guesses per second
- Modern password algorithms (bcrypt, argon2) are intentionally slow, and their cost can be raised as hardware gets faster
- In Laravel, `Hash::make()` uses the algorithm set in the configuration, and you should never write your own hash function

Rate Limiting Login Attempts

Slow hashing slows down an attacker who already has a copy of the database. An attacker just guessing through the login form needs a different defense: limiting how many attempts are allowed in a given time.

```
Route::post('/login', [LoginController::class, 'store'])  
    ->middleware('throttle:5,1');
```

- The code above allows at most 5 attempts per minute from the same source, the 6th attempt is rejected outright without even checking the password
- Just like slow hashing, the goal is to make brute-force guessing take far too long, not to prevent it completely

Session Fixation

In a session fixation attack, the attacker gives the victim a session ID the attacker already knows, before login. Once the victim logs in, that same session belongs to the attacker.

- The fix: create a new session ID right after a successful login, and discard the old one
- In Laravel, this is done with `session()->regenerate()` after `Auth::attempt()` succeeds

Part 2

Authorization and RBAC

Authorization: Permission After Identity Is Proven

Authorization: the decision of whether an authenticated user may perform one specific action on one specific resource.

- A correct identity does not mean the correct permission
- Example: an editor may edit articles, a regular reader may only read them

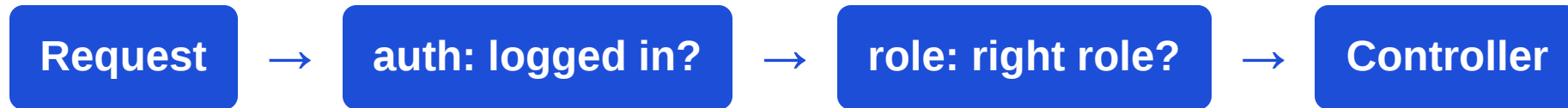
Roles, Not Individual Users



RBAC (Role-Based Access Control): permissions are attached to roles, and users get permissions through the roles they hold.

- Adding a new user only requires assigning a role, not configuring permissions one by one

Middleware as a Checkpoint



- Middleware checks a request before it reaches the controller, attached to a route or a route group
- The order matters: confirm the user is logged in first, then check the role

401 vs 403

401 Unauthorized

- No identity has been proven yet
- The fix: log in first
- In a browser, Laravel answers this by redirecting to the login page

403 Forbidden

- Identity is known, but the permission is missing
- The fix: not logging in again, but a different permission

A Route That Forgot Its Protection

Middleware only protects the routes that actually use it. A new admin route added outside the protected group can be reached by anyone who is logged in, regardless of their role.

- The mistake is invisible in testing if the tester always logs in as admin
- The prevention: also test with accounts of other roles, not only the admin account

Gate: A Simple Authorization Rule

Gate: an authorization rule written as a closure, registered under a name, and called by that name wherever it's needed.

```
Gate::define('manage-products', function (User $user) {  
    return $user->role === 'admin';  
});  
  
Gate::authorize('manage-products');
```

- Suits rules that aren't tied to one specific row of data, for example "who may open the settings page"
- Middleware checks at the route level, a Gate can be called anywhere in the code, including in the middle of a controller

Policy: Authorization per Model

Policy: a class that collects all the authorization rules for one model, one method per action (view , update , delete , etc.).

```
class ArticlePolicy
{
    public function update(User $user, Article $article): bool
    {
        return $user->id === $article->author_id;
    }
}
```

- Created via `php artisan make:policy ArticlePolicy --model=Article` , then called with `Gate::authorize('update', $article)`
- The method receives the actual row of data, so the rule can differ per row, not just by role

Middleware vs Policy: When to Use Which

Middleware

- The question: "can this reach this route at all?"
- Checked before the controller, for the whole request
- Suits role-based rules, the same for every row of data

Policy

- The question: "can this be done to this specific row?"
- Checked inside the controller, for one particular row of data
- Suits ownership rules, for example "only the original author may edit"

The `role:admin` middleware you build in the jobsheet answers the first question. Once the cashier app needs a rule like "a kasir may only void their own transaction", that's the second question, a Policy's job.

Checking Permissions in Blade: @can and @cannot

```
@can('update', $article)
    <a href="{{ route('articles.edit', $article) }}">Edit</a>
@endcan

@cannot('update', $article)
    <p>You can't edit this article.</p>
@endcannot
```

- @can / @cannot call the same Gate or Policy rule, so the rule is never written twice
- Just like @if (auth()->user()->isAdmin()) only hides the admin menu in the navigation, @can only hides the view, the real check must still happen again in the controller or route

Part 3

Package Choices and a Cross-Framework Comparison

Three Approaches in Laravel

Approach	Setup complexity	Suits
Custom middleware	Low	Simple, fixed roles, for example 2 to 3 roles
Laravel Breeze	Medium	Ready-made authentication with login and register views
Spatie Permission	Medium to high	Granular per-action permissions with many role combinations

Official docs: laravel.com/docs/authentication

When to Choose What

- Custom middleware is enough while roles are few and rarely change
- Breeze saves time building login and register pages, but you still need to understand what it generates
- Spatie Permission makes sense when permissions must be split per branch, per record, or per action
- Start with the simplest option, then move to a larger package when a real need appears

Cross-Framework Comparison

Framework	Built-in authentication	Common approach for authorization
Laravel	Auth, sessions, middleware	Middleware, Gate/Policy, packages such as Spatie
Django	Auth with users, groups, and permissions	Model-level permissions and groups
Express (Node.js)	None, uses packages	Passport.js or JWT, custom middleware
Next.js	None, uses packages	Libraries such as Auth.js (NextAuth)

Summary (1/3)

- Authentication answers "who are you", authorization answers "what are you allowed to do"; the two are checked separately
- Sessions and cookies let stateless HTTP remember a user; the cookie carries only the session ID, remember me uses a separate long-lived cookie
- Passwords are stored as hashes, and after login the session must be regenerated to prevent session fixation

Summary (2/3)

- Forgot password is solved with a one-time reset token emailed to the user, not by resending the old password
- Rate limiting (throttling) login attempts slows down brute-force guessing, just like deliberately slow hashing
- RBAC attaches permissions to roles, and middleware acts as a checkpoint at the route level before the controller

Summary (3/3)

- Use 401 for not logged in and 403 for logged in but not allowed
- A Gate answers simple rules not tied to one row of data, a Policy answers per-row rules (like ownership); `@can` / `@cannot` in Blade call the same rules
- Pick a package to fit the need: custom middleware for simple roles, Breeze for ready-made authentication, Spatie for very fine-grained permissions

References & Discussion

Official Laravel documentation (Authentication, Authorization, Middleware)

Full code: `github.com/se-polinema/simple-pos-ch07`

Next meeting: UTS (PBL Project Progress Evaluation)